

Отчёт по заданию микросервисная структура по управлению задачами – ООО «СистемаКонтроля» по работе с строительными объектами

Ефремов Алексей Игоревич ЭФБО-10-23

1. Анализ и постановка задачи

Начальный проект состоял из трёх отдельных Node.js приложений (`api_gateway`, `service_users`, `service_orders`), запускаемых через `docker-compose`. Приложения имели базовую функциональность, но не соответствовали требованиям безопасности, надёжности и удобства поддержки.

Цель работы: Цель разработки — собрать рабочий бэкенд с тремя компонентами, привести микросервисы и API-шлюз к единому стандарту качества кода, повысить надёжность, наблюдаемость и безопасность

2. Разработка и реализация

В ходе работы были выполнены следующие ключевые доработки:

- Реализована JWT-аутентификация:** В `service_users` добавлены эндпоинты `/users/register` и `/users/login`. Пароли хешируются, при успешном входе выдается JWT.
- Внедрен Middleware в API Gateway:** Шлюз теперь проверяет JWT для всех защищенных маршрутов и передает ID пользователя в заголовке `X-User-Id` в другие сервисы. Это централизует логику аутентификации.
- Реализована авторизация в сервисах:** `service_orders` использует `X-User-Id` для проверки того, что пользователь имеет доступ только к своим заказам.
- Добавлены механизмы надёжности:** В шлюз встроены Circuit Breaker (`opossum`) для отказоустойчивости и Rate Limiter для защиты от DDoS-атак.
- Улучшена трассировка:** Шлюз генерирует `X-Request-Id` для каждого запроса, что позволяет отслеживать его путь по системе.

3. Тестирование и документация

- Спецификация API:** Разработана спецификация в формате **OpenAPI 3.0** и сохранена в файле `docs/openapi.yml`. Она детально описывает все доступные эндпоинты, их параметры и ожидаемые ответы.
- Ручное тестирование:** Подготовлен файл `TESTING.md`, содержащий набор `curl` команд для пошаговой проверки всей реализованной функциональности: от регистрации до создания и получения заказа.

4. Результаты тестирования

Для проверки работоспособности системы были проведены ручные тесты с использованием `curl`, как описано в `TESTING.md`. Ниже приведены результаты ключевых тестов.

1. Регистрация пользователя

Описание: Создаем нового пользователя. Ожидаем ответ **201 Created** и данные пользователя.

Команда выполнена успешно.

```
^ ~ > curl -X POST "${API_URL}/users/register" \
-H "Content-Type: application/json" \
-d '{
  "name": "Test User",
  "email": "'"$TEST_EMAIL"'",
  "password": "'"$TEST_PASSWORD"'",
}'
{"id":1,"name":"Test User","email":"test-user-1765636192@example.com","role":"user"}%
^ ~ > |
```

2. Повторная регистрация

Описание: Пытаемся создать пользователя с тем же email. Ожидаем ответ **409 Conflict**. *Команда выполнена успешно.*

```
{ "id":1,"name":"Test User","email":"test-user-1765636192@example.com","role":"user"}%
^ ~ > curl -X POST "${API_URL}/users/register" \
-H "Content-Type: application/json" \
-d '{
  "name": "Another User",
  "email": "'"$TEST_EMAIL"'",
  "password": "anotherpassword"
}'
{"error":"User with this email already exists."}%
^ ~ > |
```

3. Вход пользователя (логин)

Описание: Используем данные для входа. Ожидаем ответ **200 OK** и JWT токен. *Команда выполнена успешно, токен получен.*

```
^ ~ > export AUTH_TOKEN=$(curl -s -X POST "${API_URL}/users/login" \
-H "Content-Type: application/json" \
-d '{
  "email": "'"$TEST_EMAIL"'",
  "password": "'"$TEST_PASSWORD"'",
}' | jq -r .token)
# Проверяем, что токен получен
echo "Auth Token: $AUTH_TOKEN"
Auth Token: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwicm9sZSI6InVzZXIiLCJpYXQiOiJlE3NjU2MzYzNDIsImV4cCI6MTc2NTYzOTk0Mn0.Y_6UCiqWlddXZhb3a3iRHZFAWAct2S3M6Cz_KrGMI4I
^ ~ > |
```

4. Доступ к защищенному маршруту без токена

Описание: Пытаемся получить профиль без токена. Ожидаем ответ **401 Unauthorized**. *Команда выполнена успешно.*

```
^ ~ > curl -I -X GET "${API_URL}/users/profile"
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Access-Control-Allow-Origin: *
RateLimit-Policy: 100;w=900
RateLimit-Limit: 100
RateLimit-Remaining: 96
RateLimit-Reset: 722
Content-Type: text/plain; charset=utf-8
Content-Length: 12
ETag: W/"c-dAuDFQrdjS3hezqxDTNgW7A0LYk"
Date: Sat, 13 Dec 2025 14:33:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

5. Доступ к профилю с валидным токеном

Описание: Получаем профиль с токеном авторизации. Ожидаем ответ **200 OK** и данные профиля. Команда выполнена успешно.

```
^ ~ > curl -X GET "${API_URL}/users/profile" \
-H "Authorization: Bearer ${AUTH_TOKEN}"

{"id":1,"name":"Test User","email":"test-user-1765636192@example.com","role":"user"}
^ ~ > |
```

6. Создание заказа

Описание: Создаем новый заказ. Ожидаем ответ **201 Created** и данные заказа. Команда выполнена успешно.

```
{
  "id": 1,
  "name": "Test User",
  "email": "test-user-1765636192@example.com",
  "role": "user"
}
^ ~ > curl -X POST "${API_URL}/orders" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${AUTH_TOKEN}" \
-d '{
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 2, "quantity": 1 }
  ]
}'

{"id":1,"userId":1,"items":[{"productId":1,"quantity":2},{"productId":2,"quantity":1}],"status":"created","createdAt":"2025-12-13T14:34:39.476Z"}
^ ~ > |
```

7. Получение списка заказов

Описание: Получаем все заказы текущего пользователя. Ожидаем **200 OK** и массив с одним заказом. Команда выполнена успешно.

```
^ ~ > curl -X GET "${API_URL}/orders" \
-H "Authorization: Bearer ${AUTH_TOKEN}"

[{"id":1,"userId":1,"items":[{"productId":1,"quantity":2},{"productId":2,"quantity":1}],"status":"created","createdAt":"2025-12-13T14:34:39.476Z"}]
^ ~ > |
```

8. Получение конкретного заказа

Описание: Получаем созданный заказ по его ID. Ожидаем **200 OK** и данные заказа. Команда выполнена успешно.

```
^ ~ > curl -X GET "${API_URL}/orders/1" \                                     © 17:35
-H "Authorization: Bearer ${AUTH_TOKEN}"
{"id":1,"userId":1,"items":[{"productId":1,"quantity":2},{"productId":2,"quantity":1}],"status":"created","createdAt":"2025-1
2-13T14:34:39.476Z"}
^ ~ > |                                                                    © 17:36
```